Robustness of Learning-Based Vulnerability Detectors under Semantics-Preserving Code Transformations

Phan The Duy[a], Author Two[a], Author Three[b]

[a] *University of Information Technology, Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam*
[b] *Institution Name, City, Country*

Abstract

Learning-based vulnerability detectors are increasingly used to support security review and software quality assurance. However, most detectors are evaluated on clean benchmark functions, while real code evolves through formatting changes, refactorings, identifier changes, and other edits that should preserve program behavior. This paper presents an empirical study of detector robustness under semantics-preserving code transformations. We construct a transformed benchmark from widely used vulnerability detection datasets, define a detector-agnostic robustness metric suite, and evaluate representative learning-based detectors under multiple transformation families. Our study quantifies prediction instability, identifies the transformations that most degrade security recall, and evaluates low-cost mitigation strategies based on prediction aggregation over transformed variants. The results provide practical guidance for evaluating and deploying AI-enabled vulnerability detection tools in software engineering workflows.

Keywords: software vulnerability detection, robustness, empirical software engineering, code transformation, software security, metamorphic testing

1. Introduction

Learning-based vulnerability detectors are increasingly used to support software security review, continuous integration pipelines, and vulnerability triage. These tools promise to help developers identify vulnerable code earlier in the software lifecycle. Their reported effectiveness, however, is commonly measured on clean benchmark functions using metrics such as accuracy, precision, recall, F1 score, or AUC. These metrics are necessary, but they do

not answer a practical software engineering question: will a detector produce stable predictions when code is changed in ways that preserve behavior?

This question matters because software rarely remains in the exact form used in benchmark datasets. Developers reformat code, rename identifiers, add comments, introduce dead branches during debugging, and refactor control flow. Such edits may change the surface form of a function without changing its behavior or its vulnerability status. A vulnerability detector that flags a vulnerable function before a harmless edit but misses the behaviorally equivalent version after the edit is brittle as a software quality assurance tool.

Listing 1 illustrates the intuition. The two functions differ syntactically: the transformed version shifts the brace layout, adds a comment, and inserts an unreachable branch. These edits should not change the behavior of the function. A robust vulnerability detector should therefore preserve its prediction across both versions.

Listing 1: Motivating example of a harmless source-level transformation.

```
int read_first(char *p) {
    return p[0];
}

int read_first(char *p)
{
    /* robustness_probe: no semantic effect */
    if (0) { int __dead = 0; (void)__dead; }
    return p[0];
}
```

This paper studies robustness under semantics-preserving code transformations as a software quality property of learning-based vulnerability detectors. Our goal is not to propose a larger detector. Instead, we provide a reproducible empirical protocol for evaluating whether existing detectors remain stable under behavior-preserving edits.

We address the following research questions:

RQ1: How much does detector performance change under semantics-preserving code transformations?

RQ2: Which transformation families cause the largest prediction instability?

RQ3: Are vulnerable functions more fragile than non-vulnerable functions under equivalent transformations?

RQ4: Can prediction aggregation over transformed variants improve robustness without retraining detectors?

This paper makes the following contributions:

1. We define a transformation-based robustness evaluation protocol for learning-based vulnerability detection.
2. We construct and validate transformed variants of widely used vulnerability detection benchmarks.
3. We empirically quantify prediction instability across detector families, transformation families, and vulnerability labels.
4. We evaluate low-cost aggregation strategies that improve robustness without retraining detectors.

2. Background and Related Work

2.1. Learning-Based Vulnerability Detection

Learning-based vulnerability detection has been studied using token-based, sequence-based, transformer-based, and graph-based representations of source code. Function-level detectors typically predict whether a function contains a vulnerability, while line-level detectors additionally localize suspicious statements. Representative systems include graph-based function-level detectors such as Devign [1] and transformer-based line-level detectors such as LineVul [2]. Recent models exploit pre-trained code representations, code property graphs, or multimodal combinations of textual and structural features. Despite this progress, many evaluations emphasize clean benchmark performance and do not explicitly measure whether predictions are invariant under harmless code edits.

2.2. Vulnerability Detection Benchmarks

Public datasets make vulnerability detection research comparable and reproducible. Common benchmarks include CodeXGLUE/Devign [3, 1], BigVul [4], DiverseVul [5], Juliet, D2A, and ReVeal. These datasets differ in size, labeling method, project diversity, and vulnerability coverage. In this pilot study, we use CodeXGLUE/Devign as the primary low-compute dataset.

Big-Vul and DiverseVul are retained as planned secondary datasets for the full journal-scale evaluation, where they will be used to assess whether the observed robustness behavior generalizes beyond CodeXGLUE/Devign.

2.3. Code Transformations and Metamorphic Testing

Metamorphic testing evaluates software by applying transformations that should preserve an expected property of the output. For vulnerability detection, a natural metamorphic relation is prediction invariance: if a transformation preserves the semantics and vulnerability status of a function, the detector prediction should remain stable. This relation is useful because it allows robustness testing even when perfect behavioral equivalence proofs are impractical for arbitrary real-world code snippets.

However, transformation validity is a central concern. Some transformations that appear semantics-preserving in isolation may change behavior in specific language contexts. We therefore use conservative transformation rules, syntax checks, and manual audits to reduce the risk of invalid variants.

2.4. Robustness of Code Intelligence Models

Prior studies show that code intelligence models can be sensitive to formatting, identifier renaming, code normalization, and other semantics-preserving edits [6]. Recent work also reports that LLM-based vulnerability detectors can be evaded by syntax- and compilation-preserving changes [7]. This work builds on that observation but focuses on vulnerability detection as a software quality assurance workflow. We define detector-oriented robustness metrics, analyze vulnerability-specific risks such as true-positive-to-false-negative flips, and evaluate low-cost mitigation strategies.

2.5. Adversarial Evasion in Security Tools

Security models have also been studied under adversarial evasion, including malware detection, phishing detection, intrusion detection, and vulnerability detection. Our study differs in emphasis. We use transformations primarily as controlled validation probes for deployed or deployable software engineering tools, not as an offensive evasion framework.

3. Study Design

3.1. Overview

Figure 1 summarizes the study pipeline. We start from benchmark functions, generate transformed variants, validate transformation applicability,

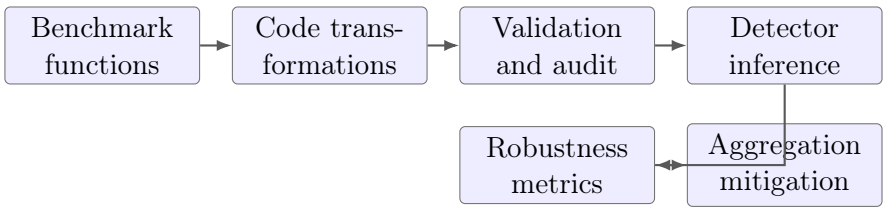

Figure 1: Overview of the transformation-based robustness evaluation pipeline.

Table 1: Prepared CodeXGLUE/Devign dataset statistics after normalization.

Split	Samples	Secure	Vulnerable	Vuln. ratio	Median chars
Train	21,854	11,836	10,018	0.458	935
Validation	2,732	1,545	1,187	0.434	928
Test	2,732	1,477	1,255	0.459	966

run vulnerability detectors on original and transformed functions, and compute robustness metrics.

3.2. Datasets

Table 1 summarizes the prepared primary dataset. The primary dataset is CodeXGLUE/Devign because it is widely used, function-level, and feasible for low-compute experiments. Secondary datasets will be used to assess whether findings generalize beyond a single benchmark.

3.3. Transformation Taxonomy

We group transformations into four families: lexical/layout transformations, comment transformations, dead-code carrier transformations, and identifier/control-flow transformations. Table 2 summarizes the initial transformation set.

The initial submission focuses on conservative transformations that are simple to validate and inexpensive to generate. If time permits, we extend the set with scope-aware identifier renaming and restricted control-flow normalizations.

3.4. Transformation Validation

We use a three-level validation policy. First, each transformation is designed with conservative preconditions. If a function does not satisfy the preconditions, the transformation is skipped. Second, transformed code is

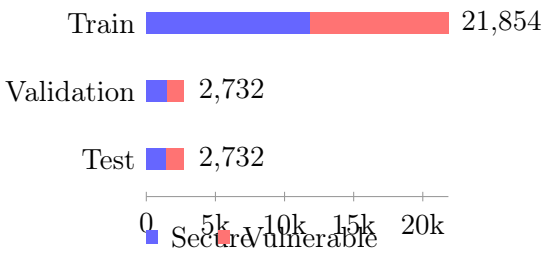

Figure 2: Class composition of the normalized CodeXGLUE/Devign splits.

Table 2: Initial transformation taxonomy and observed applicability on the CodeXGLUE/Devign test split.

ID	Transformation	Family	Expected semantic effect	Applica
T1	Blank-line expansion	Lexical/layout	None	
T2	Brace layout shift	Lexical/layout	None	
T3	Comment insertion	Comment	None	
T4	Unreachable branch insertion	Dead-code carrier	None when syntactically valid	

checked for syntax or parse validity when tooling is available. Third, we conduct a manual audit of randomly sampled transformed functions from each transformation family and dataset. The audit records whether the transformation appears to preserve behavior, whether it is syntactically valid, and whether the transformed variant should be excluded from analysis.

3.5. Detector Selection

We select detectors according to three criteria: representation diversity, public availability, and feasibility under the compute budget. The planned low-compute set includes a frozen code-model embedding classifier, an existing transformer-based vulnerability detector checkpoint, and a prompted compact code model or API model on a subset. Expanded experiments may include graph- based or line-level detectors if compute and implementation time allow.

3.6. Metrics

Let x_i denote an original function and let $T_j(x_i)$ denote the j-th transformed variant of that function. Let y_i be the ground-truth label and $\hat{y}(x)$ be the detector prediction.

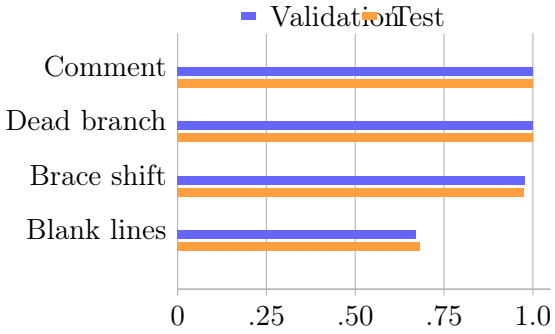

Figure 3: Transformation applicability on the CodeXGLUE/Devign validation and test splits.

Prediction flip rate.. Prediction flip rate measures the fraction of transformed variants whose prediction differs from the original prediction:

$$\text{FlipRate} = \frac{\sum_i \sum_j \mathbb{1}[\hat{y}(T_j(x_i)) \neq \hat{y}(x_i)]}{\sum_i |T(x_i)|}. \tag{1}$$

Robust accuracy.. Robust accuracy measures the fraction of clean-correct samples that remain correct under all transformed variants:

$$\text{RobustAcc} = \frac{\sum_i \mathbb{1}[\hat{y}(x_i) = y_i \wedge \forall j, \hat{y}(T_j(x_i)) = y_i]}{\sum_i \mathbb{1}[\hat{y}(x_i) = y_i]}. \tag{2}$$

Complete resistance.. Complete resistance focuses on originally detected vulnerable functions:

$$\text{CR} = \frac{\sum_{i:y_i=1} \mathbb{1}[\hat{y}(x_i) = 1 \wedge \forall j, \hat{y}(T_j(x_i)) = 1]}{\sum_{i:y_i=1} \mathbb{1}[\hat{y}(x_i) = 1]}. \tag{3}$$

We also report standard classification metrics, including accuracy, precision, recall, F1 score, and AUC when prediction scores are available.

3.7. Statistical Analysis

Because each original function is paired with transformed variants, we use paired analyses. We report bootstrap confidence intervals for performance drops and use paired tests such as McNemar's test when comparing paired prediction outcomes. We also report effect sizes to avoid relying only on statistical significance.

4. Experimental Setup

4.1. Data Preparation

All datasets are normalized to JSONL records with fields `idx`, `func`, and `target`. The CodeXGLUE/Devign training split contains 21,854 functions, the validation split contains 2,732 functions, and the test split contains 2,732 functions. The vulnerable class accounts for 45.8%, 43.4%, and 45.9% of the training, validation, and test splits, respectively. Each transformed variant keeps the original label and receives a unique `variant_id`. Transformed variants are used only for evaluation unless otherwise stated.

4.2. Transformation Implementation

Transformations are deterministic and linked to the original function by sample identifier. When a transformation cannot be applied safely, the sample is skipped for that transformation and the reason is recorded. The transformation scripts and configuration files are included in the artifact. In the prepared test split, four transformations produced 10,928 transformed rows from 2,732 original functions, for a total of 13,660 original and transformed records.

4.3. Detector Execution

For the completed low-compute pilot, we evaluate three lightweight learning-based detectors implemented locally: token multinomial Naive Bayes (*Token-NB*), character 4-gram multinomial Naive Bayes (*Char4-NB*), and hashed logistic regression over lexical tokens (*Hash-LR*). Each detector uses at most the first 8,000 characters of a function. Decision thresholds are selected on the original validation split by maximizing F1 and are then fixed for the transformed test split. This pilot is intended to validate the experimental pipeline; it is not a substitute for the stronger transformer and graph-based detectors planned for the full study.

4.4. Compute Budget

The transformation and metric computation steps run on CPU. Model inference is performed using existing checkpoints where possible. Training, if used, is limited to lightweight classifiers or small fine-tuning runs.

Table 3: Generated transformed variants for CodeXGLUE/Devign.

Split	Original records	Transformed records	Total records	Transformations
Validation	2,732	10,928	13,660	4
Test	2,732	10,928	13,660	4

Table 4: Transformation applicability on CodeXGLUE/Devign validation and test splits.

Transformation	Val. changed	Val. appl.	Test changed	Test appl.
Comment insertion	2,732	1.000	2,732	1.000
Unreachable branch insertion	2,732	1.000	2,732	1.000
Brace layout shift	2,672	0.978	2,661	0.974
Blank-line expansion	1,831	0.670	1,864	0.682

5. Results

This section reports completed dataset preparation, variant generation, validation, and low-compute pilot detector results. The results should be read as a pipeline-validating pilot rather than the final journal-scale evaluation.

5.1. Dataset and Variant Generation Results

Table 3 summarizes the transformed CodeXGLUE/Devign validation and test splits. For both splits, every original function is preserved as an `original` variant and is paired with four transformed variants. The validation and test transformed files each contain 13,660 records: 2,732 original records and 10,928 transformed records.

Table 4 reports transformation applicability. Comment insertion and unreachable branch insertion applied to all functions in both splits. Brace layout shift applied to 97.8% of validation functions and 97.4% of test functions. Blank-line expansion applied to 67.0% of validation functions and 68.2% of test functions because it requires a semicolon-newline pattern in the source text.

Figure 3 visualizes the same pattern: comment insertion and dead-branch insertion apply universally, brace layout shift applies to nearly all functions, and blank-line expansion is more dependent on the source layout.

These results establish that the first version of the robustness benchmark is ready for validation and detector inference.

Table 5: Clean and transformed performance by detector.

Detector	Clean P	Clean R	Clean F1	Worst F1	F1 drop	Robust R
Token-NB	0.473	0.948	0.631	0.630	0.001	0.944
Char4-NB	0.459	1.000	0.630	0.630	0.000	1.000
Hash-LR	0.459	1.000	0.630	0.629	0.000	0.999

5.2. Transformation Validation Results

Automated structural checks show that all transformed variants preserve their original labels. Each transformed row remains non-empty and contains an opening brace. The brace-balance rate is 97.5% for each transformation, matching the rate observed in the original snippets; this indicates that the imbalance is a property of some extracted function snippets rather than a transformation- specific artifact. We also generated a manual audit sheet containing 30 changed samples per transformation, for 120 audit rows in total. Manual semantic audit is the next quality-control step before reporting final results.

5.3. RQ1: Overall Robustness

Answer to RQ1. In this low-compute pilot, the evaluated detectors show small prediction changes under the four conservative transformations. The largest overall prediction flip rate is 0.156% for Token-NB. Char4-NB is unchanged under all evaluated variants, while Hash-LR has a 0.027% flip rate. This stability should be interpreted cautiously because Char4-NB and Hash-LR largely predict the vulnerable class for all samples, yielding high recall but low precision.

Table 5 reports clean and transformed performance. The key comparison is between clean recall/F1 and the worst transformed recall/F1.

5.4. RQ2: Transformation Sensitivity

Answer to RQ2. Comment insertion and dead-branch insertion are the only transformations that change predictions in this pilot. Blank-line expansion and brace layout shift do not change predictions for any of the three lightweight detectors.

Table 6 ranks transformations by the maximum observed sensitivity across the three pilot detectors.

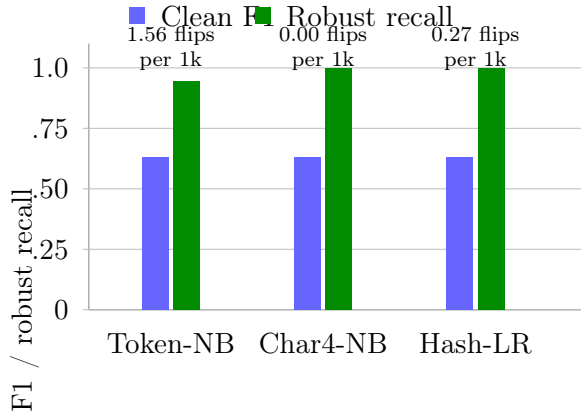

Figure 4: Clean F1, robust recall, and prediction flip rate for the low-compute pilot detectors.

Table 6: Transformation sensitivity.

Transformation	Applicable samples	Max flip	Max vuln. flip	Max F1
Blank-line expansion	1,864 changed / 2,732 total	0.000	0.000	
Brace layout shift	2,661 changed / 2,732 total	0.000	0.000	
Comment insertion	2,732 changed / 2,732 total	0.004	0.004	
Unreachable branch insertion	2,732 changed / 2,732 total	0.002	0.002	

For the largest observed effect, Token-NB under comment insertion, the recall drop is 0.004 with a bootstrap 95% confidence interval of [0.001, 0.008]. For Token-NB under dead-branch insertion, the recall drop is 0.002 with a bootstrap 95% confidence interval of [0.000, 0.005]. Paired McNemar summaries are small in this pilot because few predictions change; for comment insertion, Token-NB has five clean-only-correct and six transformed-only-correct cases.

5.5. RQ3: Label-Specific Fragility

Answer to RQ3. Token-NB shows the largest label-specific instability, with a vulnerable flip rate of 0.159% and a non-vulnerable flip rate of 0.152% across all transformed variants. The observed true-positive-to-false- negative rate is 0.159% for Token-NB and 0.020% for Hash-LR. Char4-NB shows no label-specific flips in this pilot.

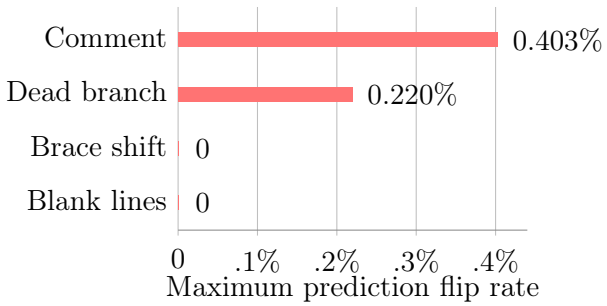

Figure 5: Maximum observed prediction flip rate by transformation across the three pilot detectors.

Table 7: Label-specific prediction instability.

Detector	Vulnerable flip	Non-vulnerable flip	TP→FN	TN→FP
Token-NB	0.0016	0.0015	0.0016	0.0000
Char4-NB	0.0000	0.0000	0.0000	0.0000
Hash-LR	0.0002	0.0003	0.0002	0.0000

This analysis distinguishes benign prediction instability from security-critical instability. In particular, true-positive-to-false-negative flips indicate cases where a detector originally identified a vulnerable function but missed a transformed equivalent.

5.6. RQ4: Mitigation through Variant Aggregation

Answer to RQ4. Majority vote and max-risk aggregation do not change the results for these pilot detectors because the individual predictions are already highly stable. Mean-score aggregation changes the precision–recall tradeoff, increasing precision while sharply reducing recall for all three detectors.

We evaluate three aggregation strategies over original and transformed variants: majority vote, mean score aggregation, and max-risk aggregation. The max-risk rule predicts vulnerable if any variant is predicted vulnerable. This strategy may improve robust recall but can increase false positives.

Table 8: Aggregation mitigation results for the low-compute pilot.

Detector	Strategy	Precision	Recall	F1
Token-NB	Original only	0.473	0.948	0.631
Token-NB	Majority vote	0.473	0.948	0.631
Token-NB	Mean score	0.570	0.482	0.522
Token-NB	Max-risk	0.473	0.948	0.631
Char4-NB	Original only	0.459	1.000	0.630
Char4-NB	Majority vote	0.459	1.000	0.630
Char4-NB	Mean score	0.537	0.438	0.483
Char4-NB	Max-risk	0.459	1.000	0.630
Hash-LR	Original only	0.459	1.000	0.630
Hash-LR	Majority vote	0.459	1.000	0.630
Hash-LR	Mean score	0.717	0.065	0.118
Hash-LR	Max-risk	0.459	1.000	0.630

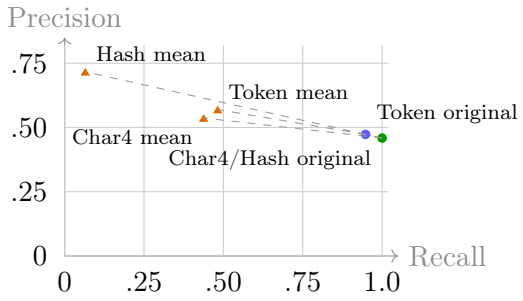

Figure 6: Precision–recall tradeoff of aggregation strategies in the low-compute pilot.

6. Discussion

6.1. Clean Accuracy Is Not Enough

Clean benchmark metrics remain important, but they do not capture prediction stability. A detector can achieve strong clean F1 while failing to preserve predictions under harmless edits. Robustness metrics should therefore be reported alongside standard metrics for vulnerability detection.

6.2. Implications for CI/CD Security Gates

Prediction instability has direct implications for security automation. If a small behavior-preserving edit converts a true positive into a false negative,

then an automated gate may allow vulnerable code to pass. Security-sensitive workflows may prefer conservative aggregation strategies that increase recall, even when they introduce additional false positives.

6.3. Implications for Benchmark Designers

Benchmark maintainers should consider releasing transformed variants, reporting transformation validity, and separating robustness test sets from training data. They should also report token lengths and truncation rates because preprocessing artifacts can affect robustness.

6.4. Implications for Tool Builders

Tool builders can use transformation-based tests as part of pre-deployment validation. Prediction instability across code review revisions can also be monitored as an operational signal. Lightweight aggregation over transformed variants is one possible mitigation when retraining is infeasible.

6.5. Possible Causes of Brittleness

Detector brittleness may arise from token-level shortcut learning, identifier memorization, sensitivity to formatting, truncation artifacts, or weak modeling of data and control dependencies. The relative importance of these causes may vary by detector family and transformation type.

7. Threats to Validity

7.1. Construct Validity

The main construct validity threat is that a transformation may not preserve semantics in all contexts. We mitigate this risk through conservative transformation rules, syntax checks, and manual audit. We also report the invalid transformation rate and exclude invalid variants from analysis.

7.2. Internal Validity

Prediction changes may be caused by tokenization, truncation, preprocessing, or thresholding rather than model reasoning. We mitigate this threat by using fixed thresholds, logging token lengths, reporting truncation rates, and applying transformations only at evaluation time.

7.3. External Validity

Function-level datasets may not fully represent whole-project vulnerability detection or deployment in industrial CI/CD systems. We mitigate this threat by using multiple datasets and by limiting claims to function-level vulnerability detection.

7.4. Conclusion Validity

Observed robustness drops may depend on sample selection or detector configuration. We mitigate this threat by using paired analyses, confidence intervals, multiple detector families, and a reproducible artifact package.

8. Conclusion

This paper argues that learning-based vulnerability detectors should be evaluated not only by clean benchmark accuracy but also by robustness under semantics-preserving code transformations. We present a low-compute empirical protocol, a transformation taxonomy, a detector-agnostic metric suite, and a mitigation analysis based on variant aggregation. The resulting evidence can help researchers, benchmark designers, and tool builders evaluate whether AI-enabled vulnerability detection tools are stable enough for software engineering workflows.

Data Availability

The pilot artifact is available locally with this manuscript and contains the transformation scripts, experiment configuration, transformed benchmark metadata samples, prediction files, robustness metric scripts, and the compiled paper. The artifact is packaged as `artifact_codexglue_pilot.zip`. Before journal submission, this package should be uploaded to a persistent public archive and this statement should be updated with the corresponding DOI or repository URL.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Advances in Neural Information Processing Systems, 2019, pp. 10197–10207.

[2] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the International Conference on Mining Software Repositories, 2022.

[3] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. B. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, S. Liu, CodeXGLUE: A machine learning benchmark dataset for code understanding and generation, arXiv preprint arXiv:2102.04664 (2021).

[4] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A c/c++ code vulnerability dataset with code changes and cve summaries, in: Proceedings of the International Conference on Mining Software Repositories, 2020.

[5] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses, 2023.

[6] Y. Li, S. Qi, C. Gao, Y. Peng, D. Lo, D. Y.-A. Lo, M. R. Lyu, Z. Xu, Understanding the robustness of transformer-based code intelligence via code transformation: Challenges and opportunities, IEEE Transactions on Software Engineering 51 (2) (2025) 521–547.

[7] L. Sun, A. Oprea, E. Wong, Syntax- and compilation-preserving evasion of llm vulnerability detectors, arXiv preprint arXiv:2602.00305 (2026).